

# Clock-Domain-Crossing IP Library

Design, Verification, Packaging and FPGA Implementation

HFT Tick-to-Trade Accelerator · **cdc\_integration** block design · 30 July 2026

This report documents a hand-written clock-domain-crossing (CDC) primitive library built for a three-clock-domain HFT tick-to-trade design, together with its verification, Vivado IP packaging, block-design integration, timing-constraint strategy, and the post-route implementation results on a Xilinx UltraScale+ device. It also records the engineering problems encountered and how each was diagnosed and resolved, since several were subtle and are worth not re-learning.

## 1. Motivation and Domain Architecture

The original tick-to-trade design ran entirely on a single clock. Real trading hardware does not: the line-side receive logic is tied to the Ethernet recovered clock, the trading core is pushed as fast as timing allows, and the host/CSR interface runs slower and asynchronously. Moving to three domains makes the design representative and forces every boundary to be handled explicitly.

| Domain     | Clock    | Period                | Role                                                        |
|------------|----------|-----------------------|-------------------------------------------------------------|
| RX-line    | clk_line | 6.400 ns (156.25 MHz) | Models a 10G MAC receive datapath; ingests raw ITCH bytes   |
| Core       | clk_core | 4.000 ns (250.00 MHz) | Parser, order book, strategy, risk checks, order emit       |
| Host / CSR | clk_host | 8.000 ns (125.00 MHz) | AXI-Lite register access, configuration, telemetry readback |

**Honest scope note.** Inside the target FPGA shell there is no real Ethernet MAC available to this design, so the RX-line clock is a *modelled* auxiliary shell clock rather than a genuine recovered clock. The CDC logic and constraints are identical either way and transfer unchanged; only the clock source differs.

The existing module boundaries were already stream-shaped — AXI-Stream bytes in, AXI-Stream decisions out, AXI-Lite for control — so they form natural, clean crossing points rather than requiring the datapath to be restructured.

## 2. The Five CDC IPs

Each block is hand-written rather than instantiated from Xilinx XPM. The XPM equivalents are documented as a production swap path, but building them by hand makes the failure modes explicit and demonstrable, which is the point of the exercise.

| IP              | Crosses     | Mechanism                           | Use for                                  |
|-----------------|-------------|-------------------------------------|------------------------------------------|
| sync_2ff        | 1 bit       | N-flop synchronizer chain           | Level-stable single-bit controls         |
| pulse_sync      | 1 bit event | Toggle + closed-loop acknowledge    | Single-cycle strobes between domains     |
| async_fifo      | Data stream | Gray-coded pointers, dual-clock RAM | Bulk data with elasticity                |
| axis_async_fifo | AXI-Stream  | async_fifo + AXIS handshake wrapper | Stream interfaces at IP boundaries       |
| handshake_mcp   | N-bit word  | Req/ack multi-cycle path            | Atomic configuration and telemetry words |

### 2.1 sync\_2ff — single-bit synchronizer

The workhorse primitive. It brings one asynchronous bit into the destination clock through a chain of STAGES flip-flops. The first flop may go metastable when its input changes near the sampling edge; the remaining flop(s) give that metastability time to resolve, so mean-time-between-failure grows exponentially with chain depth. Two stages is standard; three is used for very high-frequency or high-reliability crossings.

```
module sync_2ff #(
    parameter int STAGES = 2,      // synchronizer depth (>= 2)
    parameter bit INIT_VAL = 1'b0 // reset / power-up value of the chain
)(
    input  logic clk, rst_n,        // destination domain
    input  logic d,                 // asynchronous bit (source domain)
    output logic q                  // synchronized bit (in `clk` domain)
);
```

**Critical restriction.** Use this for SINGLE-BIT signals only. Instantiating one per bit of a bus is a classic and serious bug: each bit resolves independently, on different cycles, so the destination can momentarily observe a value that was never driven. Buses must cross via `async_fifo` (data) or `handshake_mcp` (configuration).

The chain carries the `ASYNC_REG` attribute in RTL, which tells synthesis and place-and-route that these are synchronizer flops: never merge or retime them away, and place them physically adjacent to maximise the settling window.

## 2.2 pulse\_sync — event crossing

A single-cycle pulse cannot simply be synchronized: if the destination clock is slower, the pulse can fall entirely between two sampling edges and vanish. This block converts the event into a level change instead. A source-domain toggle flips on each request; the toggle is synchronized into the destination and edge-detected to regenerate a one-cycle strobe.

A returning acknowledge toggle is synchronized back to the source, driving `src_busy`. This closes the loop: the source is told when the previous event has actually landed, so it cannot issue events faster than the destination can absorb them — the other classic pulse-crossing failure.

```
input  logic src_clk, src_rst_n, src_pulse;  output logic src_busy;
input  logic dst_clk, dst_rst_n;            output logic dst_pulse;
```

## 2.3 async\_fifo — dual-clock data FIFO

A dual-clock FIFO with a shared memory array, written in the source domain and read in the destination domain. The difficulty is not the storage but the pointer comparison: each side must know the other side's pointer to compute full and empty, and that pointer is a multi-bit value crossing a clock boundary.

The pointers are therefore **Gray coded**. In Gray code exactly one bit changes per increment, so if the destination samples mid-transition it captures either the old pointer or the new one — never an invalid intermediate. This single property is what makes a multi-bit crossing safe, and it is why the pointer bits also require a bounded-skew timing constraint (Section 6).

```
parameter int WIDTH = 8, DEPTH = 16, STAGES = 2,
           AF_LEVEL = DEPTH-2, AE_LEVEL = 1;
wr_clk / wr_rst_n / wr_en / wr_data -> full,  almost_full
rd_clk / rd_rst_n / rd_en / rd_data -> empty, almost_empty
```

The FIFO is first-word fall-through: `rd_data` presents the head of the queue without needing a read strobe first. `almost_full` and `almost_empty` provide programmable back-pressure thresholds, which matter for a line-rate ingress buffer that must signal congestion before it actually overflows.

## 2.4 axis\_async\_fifo — AXI-Stream wrapper

A thin wrapper that presents `async_fifo` as a standard AXI-Stream slave/master pair, translating between `tvalid`/`tready` handshaking and the FIFO's `full`/`empty` flags, and carrying `tlast` alongside the data. This is the form actually used at design boundaries, because it drops directly into IP Integrator and connects to any AXIS producer or consumer without glue logic.

```
s_clk / s_rst_n : s_axis_tvalid, s_axis_tready, s_axis_tdata, s_axis_tlast  
m_clk / m_rst_n : m_axis_tvalid, m_axis_tready, m_axis_tdata, m_axis_tlast
```

## 2.5 handshake\_mcp — multi-cycle-path config crossing

For a wide configuration word — a strategy selector, a risk threshold — a FIFO is overkill and per-bit synchronizers are wrong. This block uses a multi-cycle path (MCP) formulation: the source captures the word into a holding register and raises a request toggle; only that 1-bit toggle is synchronized. When the destination sees the request it samples the held data and returns an acknowledge toggle.

The wide data bus itself crosses with **no synchronizer at all**, and that is correct by construction: the source holds it constant until acknowledged, so it is guaranteed stable for the entire round trip. It needs a bounded delay, not synchronization. Vivado reports this structure as CDC-15, which is the expected signature of a properly built MCP — a genuinely unprotected bus reports CDC-1 or CDC-2 (critical) instead.

```
parameter int WIDTH = 32, STAGES = 2;  
src_clk: src_valid, src_data[WIDTH-1:0] -> src_ready  
dst_clk: dst_valid (1-cycle) , dst_data[WIDTH-1:0] (held stable between updates)
```

## 3. Functional Verification

The library was verified before any Vivado work, using cocotb 2.0.1 driving Verilator 5.036. Every block is exercised in **both clock-ratio directions** (fast source to slow destination and the reverse), because CDC bugs are frequently direction-specific — a design that works fast-to-slow can drop events slow-to-fast, and vice versa.

```
$ cd sim && make cdc-regression  
...  
==== CDC regression PASSED ==== 8 tests, 0 failures
```

Independent Python golden models in `sim/golden/` (`async_fifo.py`, `handshake_mcp.py`) provide reference behaviour, so the testbench checks against an independently written model rather than against the RTL's own assumptions.

### Verification lessons worth keeping

- **Drive stimulus on the falling edge.** Under Verilator a signal written immediately before `await RisingEdge` is not visible at that edge. Driving on the falling edge guarantees stability at the sampling edge. This initially made a 2-flop synchronizer appear to take three cycles, and stopped toggle-based blocks from firing at all.
- **Sample one-cycle strobes on the falling edge** too; sampling on the rising edge is delta-cycle sensitive and flaky.
- **Beware last-beat cancellation.** A producer that asserts valid, exits its loop and deasserts before the committing edge silently loses the final beat.
- **First-word fall-through consumers** must record the current head and assert pop for the following edge; gating on the previous ready and sampling after the pop records the wrong beat and stalls.

## 4. IP Packaging

Each block is packaged as a standalone Vivado IP with an out-of-context (OOC) synthesis mode, so it synthesizes independently and is cached. Two constraint files accompany each block, and the distinction between them is load-bearing.

| File        | Packaged?                               | Contents                                | Why                                                                         |
|-------------|-----------------------------------------|-----------------------------------------|-----------------------------------------------------------------------------|
| <block>.xdc | Yes — USED_IN synthesis, implementation | ASYNC_REG, set_max_delay -datapath_only | Names no clock and no port, so it stays valid at any depth in any hierarchy |

| File            | Packaged?                           | Contents                                 | Why                                                                             |
|-----------------|-------------------------------------|------------------------------------------|---------------------------------------------------------------------------------|
| <block>_ooc.xdc | Only with USED_IN<br>out_of_context | create_clock on the block's<br>own ports | Describes the standalone world;<br>valid only while the block is its<br>own top |

**The trap.** USED\_IN\_out\_of\_context is *exclusive*, not additive. A file carrying it does not apply in the integrated design at all — even though it still sits in the synthesis fileset. Tagging the OOC file correctly is what stops it creating shadow clocks; accidentally tagging the always-apply file the same way silently disables every CDC exception the IP carries, and no report anywhere says so.

Because that failure is invisible, a checker script (ip/check\_packaged\_ip.py) validates each packaged component .xml directly, without launching Vivado. It enforces that the always-apply XDC is shipped in the synthesis fileset and is not OOC-tagged, that the OOC file is either absent or correctly tagged, that clock interfaces carry FREQ\_HZ and ASSOCIATED\_RESET, that resets are ACTIVE\_LOW, and that every source the block instantiates actually ships. It exits non-zero so it can gate a script.

## 5. Block-Design Integration

The cdc\_top block design instantiates all five IPs — six instances, with axis\_async\_fifo appearing twice — wired across the three domains to exercise every crossing direction.

| Instance          | Source → Destination | Purpose in the real design         |
|-------------------|----------------------|------------------------------------|
| axis_async_fifo_0 | clk_line → clk_core  | ITCH byte-stream ingress           |
| axis_async_fifo_1 | clk_core → clk_host  | Decision/telemetry egress          |
| async_fifo_0      | clk_host ↔ clk_core  | Raw dual-clock buffering           |
| handshake_mcp_0   | clk_host → clk_core  | Strategy select, risk thresholds   |
| pulse_sync_0      | clk_host ↔ clk_core  | Host-issued strobes (GO, clear)    |
| sync_2ff_0        | → clk_core           | Quasi-static controls such as halt |

## 6. Timing-Constraint Strategy

This is the part of the work with the most engineering content, and the part that changed most during implementation.

### 6.1 Why not set\_clock\_groups

The obvious constraint for three unrelated clocks is:

```
set_clock_groups -asynchronous -group [get_clocks clk_line] \
                  -group [get_clocks clk_core] \
                  -group [get_clocks clk_host]
```

It works, and the design closed timing with it. But it removes the cross-domain paths from analysis **entirely**, which has two consequences. First, it overrides every set\_max\_delay, so any datapath bound written elsewhere becomes inert. Second, and more seriously, the router is left with no delay target on those nets and is free to skew a multi-bit crossing arbitrarily.

For a Gray-coded FIFO pointer that dismantles the safety argument. "Only one bit changes per increment" protects the destination only if the bits arrive within roughly one destination period of each other. With unbounded routing, a slow bit from one increment can land after a fast bit from the next, and the destination samples a pointer value that never existed — producing a wrong full/empty result and hence lost or duplicated data.

There is a second, softer argument. Under clock groups, a crossing nobody remembered to protect looks exactly as green as one that was carefully constrained, because neither is checked. Bounding the paths instead means an unconstrained crossing shows up as a timing violation. "Green because verified" is a materially stronger claim than "green because ignored".

## 6.2 What was used instead

Every ordered domain pair carries an explicit datapath bound, budgeted at one `clk_core` period — the fastest destination of the three, applied uniformly so the bound is conservative everywhere:

```
set CDC_MAX 4.000
set_max_delay -datapath_only -from [get_clocks clk_line] -to [get_clocks clk_core] $CDC_MAX
set_max_delay -datapath_only -from [get_clocks clk_core] -to [get_clocks clk_line] $CDC_MAX
set_max_delay -datapath_only -from [get_clocks clk_core] -to [get_clocks clk_host] $CDC_MAX
set_max_delay -datapath_only -from [get_clocks clk_host] -to [get_clocks clk_core] $CDC_MAX
set_max_delay -datapath_only -from [get_clocks clk_line] -to [get_clocks clk_host] $CDC_MAX
set_max_delay -datapath_only -from [get_clocks clk_host] -to [get_clocks clk_line] $CDC_MAX
```

`-datapath_only` instructs the timing engine to count only logic and routing delay and to ignore clock skew and uncertainty, which is the correct treatment for genuinely asynchronous paths. Xilinx UG903 recommends this form over clock groups and false paths on CDC nets for precisely the reasons above.

### *Why clock-to-clock rather than cell-to-cell*

An earlier revision enumerated each crossing by cell name (`wgray_reg` → `wq_reg` and so on). That fails in the out-of-context flow: while the top level is synthesized the IPs are black boxes, their internal cells do not yet exist, and every filter returns nothing:

```
[Vivado 12-4739] set_max_delay: No valid object(s) found for '-from ...'
```

Clocks exist at every stage, so a clock-to-clock bound applies during both synthesis and implementation — and as a bonus it covers every crossing between the pair, including any not enumerated by hand. Finer-grained per-crossing bounds still belong inside each IP's own XDC, where the cells are visible when that file is read.

## 7. Results

### 7.1 Post-route timing — all constraints met

| Check              | Value     | Failing endpoints | Total |
|--------------------|-----------|-------------------|-------|
| Setup (WNS)        | +2.097 ns | 0                 | 593   |
| Hold (WHS)         | +0.046 ns | 0                 | 527   |
| Pulse width (WPWS) | +1.468 ns | 0                 | 275   |

### 7.2 Per clock domain

| Clock    | Frequency  | Setup WNS | Hold WHS  | Endpoints |
|----------|------------|-----------|-----------|-----------|
| clk_core | 250.00 MHz | +2.097 ns | +0.050 ns | 365       |
| clk_line | 156.25 MHz | +5.330 ns | +0.051 ns | 95        |
| clk_host | 125.00 MHz | +6.572 ns | +0.046 ns | 67        |

### 7.3 Cross-domain crossings, bounded and met

Every pair is analyzed against the 4.000 ns datapath budget and passes with approximately 3 ns of margin:

| Source → Destination | Slack     | Failing | Endpoints |
|----------------------|-----------|---------|-----------|
| clk_host → clk_core  | +2.884 ns | 0       | 44        |
| clk_core → clk_host  | +3.009 ns | 0       | 12        |
| clk_line → clk_core  | +3.583 ns | 0       | 5         |
| clk_core → clk_line  | +3.587 ns | 0       | 5         |

## 7.4 Routing, DRC and utilization

- **Routing:** 389 of 389 routable nets fully routed, 0 nets with routing errors.
- **DRC:** no violations.
- **Utilization:** 108 CLB LUTs (36 as distributed RAM), 200 registers, 0 block RAM.
- **Device:** xcu200-fsgd2104, speed grade -2.

## 7.5 CDC report

No CDC-1 or CDC-2 (critical) findings. All entries are CDC-3 (Info, single-bit synchronized with ASYNC\_REG), CDC-6 (multi-bit synchronized with ASYNC\_REG) or CDC-15 (clock-enable controlled structure).

**CDC-6 on the FIFO pointers is expected and is waived, not fixed.** Vivado flags any multi-bit bus through a synchronizer because that is normally the classic CDC bug. Here the bus is Gray coded, so a skewed capture yields either the old or the new pointer and never an invalid intermediate; the tool cannot infer the encoding from the netlist. The same report shows the genuinely single-bit pointer MSB as CDC-3 Info through the identical synchronizer — the classification differs purely on bit width. CDC-15 on the handshake\_mcp data bus is likewise the correct signature of a multi-cycle path.

## 8. Problems Encountered and Resolutions

Several of these were subtle, cost real time, and are worth recording.

### 8.1 Vivado repeatedly killed by memory exhaustion

**Symptom:** Vivado died partway through synthesis, leaving stale `__synthesis_is_running__` markers.

**Cause:** the build machine has 16 cores but only 17 GB of RAM. Vivado sizes its default parallel job count from core count, so it launched all six out-of-context IP runs simultaneously. Each peaked around 3.6 GB — roughly 21 GB against 17 GB available. Every log ended with `free physical` under 250 MB. The memory is almost entirely the UltraScale+ device database, not the user logic: a trivial two-flop synchronizer peaked at 3.1 GB, essentially the same as the largest block.

**Resolution:** run the OOC synthesis runs strictly serially (`launch_runs -jobs 1` with `wait_on_run` between each) and run headless so the GUI's own 1–2 GB is not competing. Full build time was about 13 minutes. Note that the GUI's *Run Synthesis* button relaunches every out-of-date run at its own default job count and ignores any previously set value, so the serial script is the only reliable path on this machine.

### 8.2 FIFO memory inferred as flip-flops instead of RAM

**Symptom:** [Synth 8-4767] Trying to implement RAM 'mem\_reg' in registers. Block RAM or DRAM implementation is not possible — RAM is sensitive to asynchronous reset signal.

**Cause:** the FIFO array write sat inside an `always_ff` block that carried an asynchronous reset. Block RAM and LUTRAM primitives have no asynchronous reset on the array, so synthesis could not map it and fell back to discrete registers.

**Resolution:** move the array write into its own reset-free `always_ff @(posedge wr_clk)` block, which is the canonical inference template. The storage now maps to distributed RAM (36 LUTs, visible in the final utilization).

At this FIFO depth the difference is small — roughly 512 bits — but it would dominate area and timing at production depths.

### 8.3 CDC verified against the wrong clocks

**Symptom:** `report_cdc` listed crossings between clocks named `wr_clk`, `s_clk` and `src_clk` — the IPs' own port names — rather than `clk_line`, `clk_core` and `clk_host`.

**Cause:** each packaged IP shipped an OOC constraint file performing `create_clock` on the block's own ports. Once the IP is instantiated those ports are internal pins, and the clocks became shadow copies competing with the real top-level definitions. Three of the six instances were being analyzed against these shadow clocks and therefore were never verified at their actual frequencies, while the report still looked clean.

**Resolution:** tag the OOC file `USED_IN out_of_context` so it applies only during the IP's own standalone run. This is also the origin of the packaging checker described in Section 4 — the failure was invisible in every report, so it needed a dedicated test.

### 8.4 Two constraint gaps hidden by the clock groups

Removing `set_clock_groups` exposed two crossings that had never actually been constrained. Neither was visible while the paths were excluded from analysis.

- **The Gray pointer MSB was unconstrained.** The Gray-code MSB equals the binary MSB, so synthesis drives that bit straight from the binary counter and no corresponding `gray_reg` cell exists. A filter matching only `*gray_reg*` silently missed one bit of every pointer bus — precisely the bit whose skew the constraint exists to bound.
- **The handshake\_mcp request and acknowledge toggles had no delay bound at all**, only the `ASYNC_REG` attribute.

Both are fixed in the IP-level constraint files. This is the clearest practical argument for bounding rather than ignoring: switching the constraint style is what surfaced them.

### 8.5 XDC language restrictions

Two Vivado behaviours are worth stating plainly, because both fail quietly:

- An `.xdc` accepts constraint commands plus basic `set/expr` only. `proc`, `puts`, `if` and `foreach` are rejected with `[Designutils 20-1307]` — and Vivado then skips them and continues, so the run appears to succeed with the constraints simply absent. Real Tcl must live in a `.tcl` script.
- A top-level XDC cannot reference cells inside an out-of-context IP, because those cells do not exist while the top level is synthesized. The accompanying `[Project 1-486]` `Could not resolve non-primitive black box cell` messages are normal for the OOC flow and are not errors.

## 9. Known Gaps and Next Steps

- **Re-package the IPs so the library is self-sufficient.** The CDC bounds currently live in the top-level constraint file. For the blocks to be genuinely reusable, each IP should carry its own exceptions with the correct `USED_IN` setting.
- **Add `set_bus_skew` to the pointer buses.** `report_bus_skew` currently reports no bus-skew constraints. `set_max_delay` bounds each bit's absolute delay, which bounds skew indirectly; `set_bus_skew` states the relative-skew requirement directly and is what Xilinx's own `XPM_CDC` uses alongside `max_delay`.
- **Phase 2 integration.** Build the multi-clock tick-to-trade top level, placing these blocks on real datapaths rather than the present scaffold. Note that the current block design drives every IP input from a top-level port, so it validates each IP's *internal* CDC structure; system-level crossings can only be verified once real logic drives them.

- **Deepen the FIFOs.** Depth is at the default 16. A line-rate ingress buffer needs considerably more, and that is also where the RAM-inference fix of Section 8.2 starts to matter materially.

---

Generated 30 July 2026 · Vivado 2025.2 · Verilator 5.036 · cocotb 2.0.1 · device xcu200-fsgd2104